

Dokumentation der Vermessungsmethoden

Vermessung von Fingern anhand dreidimensionaler Scans

Technische und mathematische Dokumentation

Datengrundlage:	3D-Scans als OBJ-Dateien
Programmiersprache:	Python
3D-Verarbeitung:	PyVista / NumPy / VTK
Texturdaten:	MTL / JPG

1. September 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Datenbasis und technische Grundlagen	3
2.1	Dreidimensionaler Scan	3
2.2	OBJ-Dateien	3
2.3	MTL-Dateien und Texturen	4
3	Aufbau der Vermessungsfunktionen	4
4	Messung einer Strecke auf der Oberfläche	4
4.1	Bedienung	4
4.2	Geodätischer Pfad	5
4.2.1	Kontinuierliche Beschreibung einer Kurve	5
4.2.2	Diskrete Darstellung durch das Mesh	5
4.2.3	Zusammenhang zwischen kontinuierlicher und diskreter Längenberechnung	6
5	Flächeninhalt und Umfang	6
5.1	Flächeninhalt	6
5.2	Umfang einer geschlossenen Punktfolge	6
6	Volumenberechnung	7
7	Mathematische Grundlage der Volumenberechnung	7
7.1	Volumen einer geschlossenen Oberfläche	7
7.2	Diskrete Darstellung durch Dreiecke	7
8	Verschließen des offenen Meshes	8
8.1	Erkennen des offenen Randes	8
8.2	Bestimmung der Randpunktmenge	8
8.3	Bestimmung der Deckelebene mit PCA	8
8.4	Sortierung der Randpunkte	8
8.5	Orientierung des Deckels	9
8.6	Erzeugung des Deckels	9
9	Volumen des vollständigen Meshes	9
10	Erzeugung geschlossener Schnittkörper	10
11	Volumen ab einer Markierung	10
11.1	Bestimmung der Schnitthöhe	10
11.2	Mathematische Interpretation	11
12	Erkennung farblich markierter Bereiche	11
13	Volumen ab einer Ringmarkierung	11
13.1	Herleitung der PCA-Ebene	12
13.2	Orientierung der Normalen	12
14	Volumenberechnung ab der Ringebene	12
15	Volumen ab der Zwischenfingerfalte	12

16 Mathematischer Gesamtzusammenhang	13
17 Fehlerbetrachtung	13
17.1 Mesh-Auflösung	13
17.2 Scanfehler	13
17.3 Fehler bei der Punktwahl	13
17.4 Farbtoleranz	13
17.5 Fehler bei der Bestimmung der Schnittebene	14
17.6 Fehler beim Verschließen des Meshes	14
17.7 Geschlossenheit des Meshes	14
18 Zusammenfassung der Verfahren	14
19 Fazit	14

1 Einleitung

Diese Dokumentation beschreibt die im Fingerscan-Viewer implementierten Verfahren zur geometrischen Vermessung dreidimensionaler Fingerscans. Die Vermessungen werden direkt auf der aus einem Scan erzeugten Polygoneometrie durchgeführt. Neben der Beschreibung der Implementierung wird erläutert, auf welchen mathematischen Grundlagen die einzelnen Messverfahren beruhen.

Die Vermessung erfolgt vollständig auf der dreidimensionalen Geometrie, da eine einfache euklidische Distanz bei einem gekrümmten Finger nicht der Strecke entlang der Oberfläche entspricht.

Die implementierten Verfahren umfassen:

- Messung einer Strecke auf der Oberfläche,
- Bestimmung von Flächeninhalt und Umfang,
- Volumenberechnung des vollständigen Meshes,
- Volumenberechnung ab einer farblich markierten Höhe,
- Volumenberechnung ab einer markierten Ringstruktur,
- Volumenberechnung ab der Zwischenfingerfalte.

Der vollständige Quellcode des Projekts kann im folgenden Repository eingesehen werden:

[GitHub Repository öffnen](#)

2 Datenbasis und technische Grundlagen

2.1 Dreidimensionaler Scan

Die Grundlage der Berechnungen bildet ein dreidimensionaler Scan eines Fingers beziehungsweise eines Handbereichs. Der Scan wird als Polygonnetz gespeichert, bestehend aus einer endlichen Anzahl von Punkten und Flächen.

Ein Punkt des Meshes wird durch einen Vektor

$$P_i = \begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix} \in \mathbb{R}^3$$

beschrieben. Die Oberfläche des realen Objekts wird durch viele miteinander verbundene Polygone approximiert, die als Dreiecke vorliegen oder entsprechend trianguliert werden können.

2.2 OBJ-Dateien

Die geometrischen Daten des Scans werden in einer OBJ-Datei gespeichert, die die Koordinaten der Vertices sowie die Definition der Flächen enthält. Vereinfacht kann ein Mesh als

$$\mathcal{M} = (V, F)$$

aufgefasst werden, wobei V die Menge der Vertices und F die Menge der Flächen bezeichnet.

2.3 MTL-Dateien und Texturen

Die MTL-Datei enthält Materialinformationen, die von der OBJ-Datei referenziert werden können. Über Texturkoordinaten wird festgelegt, welcher Punkt des Bildes einem Vertex des 3D-Modells entspricht. Eine Texturkoordinate wird durch (u, v) beschrieben, wobei üblicherweise $0 \leq u, v \leq 1$ gilt.

3 Aufbau der Vermessungsfunktionen

Die Vermessung ist auf mehrere Python-Dateien aufgeteilt:

Datei	Aufgabe
<code>userinterface.py</code>	Steuerung der Benutzeroberfläche
<code>messungen.py</code>	Geometrische Berechnungen
<code>farberkennung.py</code>	Erkennung farblich markierter Punkte

Tabelle 1: Aufteilung der Vermessungsfunktionen

Die Benutzeroberfläche übernimmt die Interaktion mit dem Benutzer, während die eigentlichen geometrischen Berechnungen in `messungen.py` gekapselt sind.

4 Messung einer Strecke auf der Oberfläche

4.1 Bedienung

Bei der Streckenmessung werden zwei Punkte auf der Oberfläche des angezeigten Fingers ausgewählt. Die beiden Punkte dienen als Start- und Endpunkt der Messung.

```

1 hand_mesh = self.aktuelles_hand_mesh
2 punkte_indices = []
3
4 def punkt_geklickt(punkt, picker):
5     punkte_indices.append(
6         hand_mesh.find_closest_point(punkt)
7     )
8
9     if len(punkte_indices) == 2:
10        self.plotter.disable_picking()
11
12        pfad = hand_mesh.geodesic(
13            punkte_indices[0],
14            punkte_indices[1]
15        )
16
17        distanz = np.linalg.norm(
18            np.diff(pfad.points, axis=0),
19            axis=1
20        ).sum()
21
22        self.hinweis_label.setText(
23            f"Strecke auf der Oberfläche: "
24            f"{distanz:.1f} mm"
25        )

```

Zunächst werden die beiden vom Benutzer ausgewählten Positionen auf die nächstgelegenen Vertices des Meshes abgebildet:

```
1 hand_mesh.find_closest_point(punkt)
```

Dadurch werden nicht die exakten Mauskoordinaten verwendet, sondern Punkte, die tatsächlich Teil des Meshes sind, und zwar diejenigen, die am nächsten zum Mausklick sind.

4.2 Geodätischer Pfad

Nachdem die beiden Messpunkte ausgewählt wurden, wird zwischen ihnen ein geodätischer Pfad auf dem Polygonnetz bestimmt. Die direkte euklidische Distanz zwischen zwei Punkten A und B ist gegeben durch

$$d(A, B) = \|B - A\|_2.$$

Diese Verbindung entspricht einer geraden Strecke durch den dreidimensionalen Raum. Bei einem gekrümmten Finger kann diese Strecke jedoch durch das Innere des Fingers oder außen entlang verlaufen und damit nicht der gesuchten Strecke auf der Oberfläche entsprechen.

4.2.1 Kontinuierliche Beschreibung einer Kurve

Eine Kurve im dreidimensionalen Raum kann mathematisch durch eine Abbildung

$$\gamma : [a, b] \rightarrow \mathbb{R}^3$$

beschrieben werden. Die beiden Messpunkte entsprechen den Anfangs- und Endpunkten der Kurve:

$$\gamma(a) = A, \quad \gamma(b) = B.$$

Die Länge einer solchen Kurve wird durch das Kurvenlängenintegral bestimmt:

$$L(\gamma) = \int_a^b \|\gamma'(t)\|_2 \, dt.$$

Eine geodätische Kurve ist eine Kurve, die auf der Oberfläche verläuft und lokal eine kürzest mögliche Verbindung zwischen benachbarten Punkten darstellt.

4.2.2 Diskrete Darstellung durch das Mesh

Das Programm arbeitet nicht mit einer mathematisch kontinuierlichen Oberfläche. Grund dafür ist, dass nur endlich viele Vertices durch den Handscanner erstellt werden, bei der Vectrakamera beispielsweise ungefähr 50 Tausend pro Hand. Der von PyVista berechnete geodätische Pfad wird durch eine endliche Folge von Punkten $P_0, P_1, \dots, P_n$ repräsentiert. Die Länge jedes einzelnen Teilstücks wird mit der euklidischen Norm bestimmt:

$$l_i = \|P_{i+1} - P_i\|_2.$$

Die gesamte Länge des diskreten Pfades ergibt sich durch Aufsummieren aller Teilstücke:

$$L_n = \sum_{i=0}^{n-1} \|P_{i+1} - P_i\|_2.$$

Genau diese Berechnung wird im Programm durchgeführt:

```

1 distanz = np.linalg.norm(
2 np.diff(pfad.points, axis=0),
3 axis=1
4 ).sum()
```

Der Aufruf `np.diff(pfad.points, axis=0)` bildet die Differenzen aufeinanderfolgender Pfadpunkte. Durch `np.linalg.norm(..., axis=1)` wird die Länge jedes Vektors berechnet. Die anschließende Summation liefert die Gesamtlänge.

4.2.3 Zusammenhang zwischen kontinuierlicher und diskreter Längenberechnung

Die diskrete Berechnung kann als Approximation des Kurvenlängenintegrals verstanden werden. Gehen die Abstände zwischen den aufeinanderfolgenden Pfadpunkten gegen Null, gilt unter geeigneten Voraussetzungen

$$\lim_{n \rightarrow \infty} L_n = L(\gamma).$$

Die verwendete Summe stellt somit eine numerische Approximation der mathematischen Kurvenlänge dar. Die Genauigkeit hängt von der Qualität und Auflösung des zugrunde liegenden Meshes ab.

5 Flächeninhalt und Umfang

5.1 Flächeninhalt

Die Funktion `berechne_flaeche_und_umfang` bestimmt zunächst den Flächeninhalt des übergebenen Meshes:

```

1 flaecheninhalt = flaeche.area
```

Ein Dreieck mit den Eckpunkten A , B und C besitzt im dreidimensionalen Raum die Fläche

$$A_{\Delta} = \frac{1}{2} \|(B - A) \times (C - A)\|_2.$$

Für ein Mesh aus m Dreiecken ergibt sich somit

$$A_{\mathcal{M}} = \sum_{k=1}^m A_{\Delta,k}.$$

5.2 Umfang einer geschlossenen Punktfolge

Die übergebenen Punkte bilden eine geschlossene Kurve. Dazu wird der erste Punkt am Ende der Liste erneut eingefügt. Für die Punktfolge $P_0, P_1, \dots, P_n$ wird die Folge $P_0, P_1, \dots, P_n, P_0$ erzeugt. Der Umfang ergibt sich daraus zu

$$U = \sum_{i=0}^{n-1} \|P_{i+1} - P_i\|_2 + \|P_0 - P_n\|_2.$$

6 Volumenberechnung

Die Volumenmessungen bilden den technisch umfangreichsten Teil der Vermessungsfunktionen. Ein zentraler Punkt ist, dass ein 3D-Scan an seinem offenen Ende keine vollständig geschlossene Oberfläche besitzen muss. Daher werden zwei Hilfsfunktionen verwendet:

- `schliesse_offenes_ende()` verschließt ein bereits vorhandenes offenes Ende des Meshes;
- `schneide_und_deckle()` führt einen Schnitt durch das Mesh, erzeugt die Schnittfläche und fügt diese dem Fingerscan zu, und zwar auf das hergestellte Loch.

7 Mathematische Grundlage der Volumenberechnung

7.1 Volumen einer geschlossenen Oberfläche

Die Volumenbestimmung basiert auf dem Divergenztheorem. Für einen Körper V mit geschlossener Oberfläche ∂V gilt

$$\iiint_V \nabla \cdot \mathbf{F} dV = \iint_{\partial V} \mathbf{F} \cdot \mathbf{n} dA.$$

Wählt man das Vektorfeld

$$\mathbf{F}(\mathbf{x}) = \frac{1}{3} \mathbf{x},$$

so gilt $\nabla \cdot \mathbf{F} = 1$ und damit

$$\boxed{\text{Vol}(V) = \frac{1}{3} \iint_{\partial V} \mathbf{x} \cdot \mathbf{n} dA.}$$

7.2 Diskrete Darstellung durch Dreiecke

Für ein Dreieck mit Eckpunkten A , B und C ist der orientierte Flächenvektor

$$\mathbf{A} = \frac{1}{2}(B - A) \times (C - A).$$

Das Oberflächenintegral aus Gleichung (7.1) kann auf dem diskreten Dreiecksnetz durch eine Summe über alle Dreiecke ausgewertet werden:

$$\text{Vol}(V) \approx \frac{1}{3} \sum_{i=1}^N \mathbf{x}_{s,i} \cdot \mathbf{A}_i,$$

wobei $\mathbf{x}_{s,i}$ den Schwerpunkt des jeweiligen Dreiecks und $\mathbf{A}_i$ dessen orientierten Flächenvektor bezeichnet.

8 Verschließen des offenen Meshes

8.1 Erkennen des offenen Randes

Die Funktion `schliesse_offenes_ende` sucht zunächst nach Randkanten:

```
1 rand = mesh.extract_feature_edges(
2   boundary_edges=True,
3   feature_edges=False,
4   non_manifold_edges=False,
5   manifold_edges=False
6 )
```

Sind keine Randpunkte vorhanden, wird das Mesh unverändert zurückgegeben. Das Verfahren geht davon aus, dass das relevante Mesh höchstens ein offenes Ende besitzt.

8.2 Bestimmung der Randpunktmenge

Die gefundenen Randpunkte werden auf eindeutige Punkte reduziert:

```
1 rand_punkte = np.unique(
2   rand.points,
3   axis=0
4 )
```

8.3 Bestimmung der Deckelebene mit PCA

Zur Bestimmung der Ebene wird eine Hauptkomponentenanalyse (Principal Component Analysis) verwendet. Der Schwerpunkt der Randpunkte lautet

$$\bar{P} = \frac{1}{N} \sum_{i=1}^N P_i.$$

Die Punkte werden zentriert: $Q_i = P_i - \bar{P}$. Aus der Singulärwertzerlegung $Q = U\Sigma V^T$ wird die Richtung mit der geringsten Varianz als Normalenvektor der Ebene verwendet. Die resultierende Ebene lautet

$$E = \left\{ x \in \mathbb{R}^3 \mid \mathbf{n} \cdot (x - \bar{P}) = 0 \right\}.$$

8.4 Sortierung der Randpunkte

Damit aus den Randpunkten eine zusammenhängende Fläche erzeugt werden kann, müssen sie entlang des Randes geordnet werden. Für jeden Randpunkt wird ein Winkel mit der Funktion `arctan2` bestimmt und die Randpunkte werden nach diesen Winkeln sortiert:

```
1 relativ = rand_punkte - schwerpunkt
2
3 winkel = np.arctan2(
4   relativ @ quer,
5   relativ @ referenz
6 )
7
8 geordnete_punkte = rand_punkte[
9   np.argsort(winkel)
10 ]
```

8.5 Orientierung des Deckels

Für die Volumenberechnung ist auch die Orientierung der Dreiecke entscheidend. Die Flächennormale eines Dreiecks mit den Punkten A , B und C ist proportional zu $(B - A) \times (C - A)$. Vertauscht man zwei Punkte, ändert sich das Vorzeichen.

Im Programm wird die Orientierung überprüft:

```

1 mesh_schwerpunkt = mesh.points.mean(axis=0)
2
3 nach_aussen_erwartet = (
4     schwerpunkt - mesh_schwerpunkt
5 )
6
7 test_normale = np.cross(
8     geordnete_punkte[1] - geordnete_punkte[0],
9     geordnete_punkte[2] - geordnete_punkte[0]
10 )
11
12 if np.dot(
13     test_normale,
14     nach_aussen_erwartet
15 ) < 0:
16     geordnete_punkte = geordnete_punkte[::-1]
```

Ist das Skalarprodukt negativ, wird die Reihenfolge der Randpunkte umgekehrt.

8.6 Erzeugung des Deckels

Der Schwerpunkt der Randpunktwolke wird als zusätzlicher Punkt in der Mitte des Deckels verwendet. Anschließend wird jedes Randsegment mit diesem Mittelpunkt verbunden, wodurch ein fächerförmiger Deckel aus Dreiecken entsteht. Für die Randpunkte $P_0, P_1, \dots, P_{N-1}$ und den Mittelpunkt C entstehen die Dreiecke (P_i, P_{i+1}, C) mit zyklischem Index. Das ursprüngliche Mesh und der Deckel werden anschließend vereinigt und trianguliert:

```

1 return mesh.merge(deckel).triangulate()
```

9 Volumen des vollständigen Meshes

Die eigentliche Funktion für das Gesamtvolumen ist bewusst kurz:

```

1 def volumen_gesamtes_mesh(
2     hand_mesh: p_v.PolyData
3 ) -> float:
4
5     geschlossen = schliesse_offenes_ende(
6         hand_mesh
7     )
8
9     return geschlossen.volume
```

Der Ablauf lässt sich zusammenfassen als

Scan-Mesh

- offenen Rand erkennen
- Deckelebene bestimmen
- Deckel erzeugen
- geschlossenes Mesh
- Volumen

Der erzeugte Deckel stellt eine geometrische Annahme dar. Es wird angenommen, dass der offene Rand durch eine annähernd planare Fläche geschlossen werden kann.

10 Erzeugung geschlossener Schnittkörper

Auch bei den Volumenmessungen nach einer Schnittebene muss darauf geachtet werden, dass der resultierende Teilkörper geschlossen ist. Dafür wurde die Funktion `schneide_und_deckle` eingeführt.

Eine Ebene kann durch einen Punkt $\mathbf{o}$ und einen Normalenvektor $\mathbf{n}$ beschrieben werden:

$$E = \left\{ x \in \mathbb{R}^3 \mid \mathbf{n} \cdot (x - \mathbf{o}) = 0 \right\}.$$

Im Programm wird diese Ebene mit VTK erzeugt und als Clipping-Ebene verwendet:

```

1 clipper = vtk.vtkClipClosedSurface()
2
3 clipper.SetInputData(mesh)
4 clipper.SetClippingPlanes(ebenen_sammlung)
5 clipper.SetGenerateFaces(True)
6
7 clipper.Update()
```

Besonders relevant ist `clipper.SetGenerateFaces(True)`. Dadurch wird die durch die Schnittebene entstehende Schnittfläche explizit als neue Geometrie erzeugt.

11 Volumen ab einer Markierung

11.1 Bestimmung der Schnitthöhe

Aus den farblich markierten Punkten wird der Mittelwert ihrer z -Koordinaten gebildet:

$$z_{\text{Schnitt}} = \frac{1}{N} \sum_{i=1}^N z_i.$$

Dadurch entsteht eine horizontale Referenzebene. Anschließend wird die Hilfsfunktion `schneide_und_deckle()` mit dem Normalenvektor $(0, 0, 1)$ aufgerufen. Das resultierende geschlossene Teilmesh kann über `geschnitten.volume` ausgewertet werden.

11.2 Mathematische Interpretation

Die Schnittebene teilt den Körper V in zwei Teilkörper. Für eine horizontale Ebene bei $z = z_0$ kann der relevante Teilkörper beschrieben werden als

$$V_{\geq z_0} = \{(x, y, z) \in V \mid z \geq z_0\}.$$

Das gesuchte Volumen ist

$$\text{Vol}(V_{\geq z_0}) = \iiint_{V_{\geq z_0}} 1 \, dV.$$

12 Erkennung farblich markierter Bereiche

Ein Teil der Vermessungsfunktionen verwendet eine farbliche Markierung im Texturbild. Die gewünschte Farbe wird aus dem Hexadezimalformat in einen RGB-Vektor umgewandelt:

$$\mathbf{c}_0 = (R, G, B)^T.$$

Für einen Vertex mit Texturkoordinaten (u, v) und ein Bild mit Breite W und Höhe H werden die Pixelkoordinaten bestimmt durch

$$x = \lfloor u(W - 1) \rfloor, \quad y = \lfloor (1 - v)(H - 1) \rfloor.$$

Der Abstand wird als euklidische Distanz im RGB-Raum definiert:

$$d(\mathbf{c}, \mathbf{c}_0) = \|\mathbf{c} - \mathbf{c}_0\|_2.$$

Ein Punkt wird als markiert betrachtet, wenn $d(\mathbf{c}, \mathbf{c}_0) < \tau$, wobei τ die Farbtoleranz bezeichnet.

13 Volumen ab einer Ringmarkierung

Eine Ringmarkierung kann beliebig im Raum orientiert sein. Daher wird aus den markierten Punkten eine Ebene bestimmt, welche die Punktwolke möglichst gut approximiert. Hierfür wird die Hauptkomponentenanalyse (PCA) verwendet.

Der Schwerpunkt der Punktwolke ist

$$\bar{P} = \frac{1}{N} \sum_{i=1}^N P_i.$$

Die Punkte werden zentriert: $Q_i = P_i - \bar{P}$. Anschließend wird eine Singulärwertzerlegung durchgeführt:

$$Q = U \Sigma V^T.$$

Die letzte Hauptachse wird als Normalenvektor der Ausgleichsebene verwendet: $\mathbf{n} = v_3$.

13.1 Herleitung der PCA-Ebene

Eine Ebene durch den Schwerpunkt $\bar{P}$ kann durch einen Einheitsnormalenvektor $\mathbf{n}$ beschrieben werden:

$$E = \left\{ x \in \mathbb{R}^3 \mid \mathbf{n} \cdot (x - \bar{P}) = 0 \right\}.$$

Der Abstand eines Punktes P_i zur Ebene ist $d_i = \mathbf{n} \cdot (P_i - \bar{P})$. Gesucht ist eine Ebene, für welche die Summe der quadrierten Abstände minimal wird:

$$\min_{\|\mathbf{n}\|_2=1} \sum_{i=1}^N \left(\mathbf{n} \cdot (P_i - \bar{P}) \right)^2.$$

Mit der Kovarianzmatrix $C = \frac{1}{N} Q^T Q$ kann die Summe geschrieben werden als $N \mathbf{n}^T C \mathbf{n}$. Nach dem Rayleigh-Ritz-Prinzip wird dieses Minimum durch den Eigenvektor zum kleinsten Eigenwert der Kovarianzmatrix erreicht. Die PCA liefert genau diese Hauptachsen.

13.2 Orientierung der Normalen

Im Programm wird die Richtung der Normalen vereinheitlicht:

```
1 if normale[2] < 0:
2     normale = -normale
```

Die Normalenvektoren $\mathbf{n}$ und $-\mathbf{n}$ beschreiben dieselbe geometrische Ebene. Die Vereinheitlichung sorgt für eine konsistente Orientierung beim anschließenden Clipping.

14 Volumenberechnung ab der Ringebene

Nachdem Schwerpunkt und Normalenvektor bestimmt wurden, wird die Schnittebene festgelegt:

$$E = \left\{ x \in \mathbb{R}^3 \mid \mathbf{n} \cdot (x - \bar{P}) = 0 \right\}.$$

Das Mesh wird anschließend mit `schneide_und_deckle()` geschnitten. Die Funktion erzeugt dabei gleichzeitig die durch die Schnittebene entstehende neue Fläche. Das resultierende Volumen kann somit auf dem geschlossenen Teilkörper bestimmt werden.

15 Volumen ab der Zwischenfingerfalte

Für diese Messung wird ein zuvor isolierter und entsprechend ausgerichteter Finger benötigt. Die Ausrichtung definiert die Ebene $z = 0$ als Referenzebene der Zwischenfingerfalte.

Vor der Berechnung wird überprüft, ob das Mesh diese Ebene überhaupt schneidet. Dazu werden die z -Grenzen des Meshes betrachtet:

$$z_{\min} = \text{bounds}_z^{\min}, \quad z_{\max} = \text{bounds}_z^{\max}.$$

Damit $z = 0$ das Mesh schneiden kann, muss gelten

$$z_{\min} \leq 0 \leq z_{\max}.$$

Der eigentliche Schnitt erfolgt mit

```

1 geschnitten = schneide_und_deckle(
2   hand_mesh,
3   normal=(0, 0, 1),
4   origin=(0, 0, 0)
5 )

```

Auch hier wird die Schnittfläche durch `schneide_und_deckle()` erzeugt, bevor das Volumen bestimmt wird.

16 Mathematischer Gesamtzusammenhang

Die bisher beschriebenen Verfahren haben eine gemeinsame Grundlage: Das reale Objekt wird durch eine diskrete Approximation beschrieben. Sei S die reale Oberfläche des Fingers und S_h die durch das Mesh approximierte Oberfläche. Für eine zunehmende Verfeinerung des Meshes gilt idealisiert $h \rightarrow 0$ und $S_h \rightarrow S$. Damit können auch die daraus bestimmten geometrischen Größen gegen die entsprechenden Größen des realen Objektes konvergieren.

Die mathematischen Verfahren liefern eine numerische Approximation der geometrischen Größen. Die tatsächliche Genauigkeit wird wesentlich durch die Qualität des Scans und des daraus erzeugten Meshes bestimmt.

17 Fehlerbetrachtung

17.1 Mesh-Auflösung

Ein reales Objekt besitzt eine kontinuierliche Oberfläche. Das verwendete Mesh besteht nur aus endlich vielen Polygonen. Bei einer groben Auflösung können gekrümmte Bereiche nur unzureichend dargestellt werden. Eine feinere Mesh-Auflösung reduziert diesen Diskretisierungsfehler im Allgemeinen.

17.2 Scanfehler

Bereits der Scanvorgang kann Abweichungen verursachen, wie Messrauschen, fehlende Oberflächenbereiche, lokale Verzerrungen oder Ungenauigkeiten bei der Rekonstruktion. Solche Fehler werden von den nachfolgenden Berechnungen übernommen.

17.3 Fehler bei der Punktwahl

Bei der Streckenmessung wird der angeklickte Punkt auf den nächstgelegenen Mesh-Vertex abgebildet. Ist der tatsächliche Klickpunkt P und der verwendete Vertex P_h , entsteht ein Fehler

$$e = \|P - P_h\|_2.$$

17.4 Farbtoleranz

Bei der Markierungserkennung wird eine Toleranz τ verwendet. Eine kleine Toleranz kann dazu führen, dass markierte Punkte nicht erkannt werden. Eine zu große Toleranz kann auch Pixel erfassen, die nicht zur eigentlichen Markierung gehören.

17.5 Fehler bei der Bestimmung der Schnittebene

Bei der Volumenberechnung ab einer Markierung wird die Schnitthöhe durch den Mittelwert der z -Koordinaten bestimmt. Enthält die erkannte Punktmenge fehlerhafte Punkte, kann sich die Position der Schnittebene verschieben. Bei der Ringmarkierung hängt die Genauigkeit der PCA-Ebene von der Qualität und Verteilung der markierten Punkte ab.

17.6 Fehler beim Verschließen des Meshes

Das Verschließen eines offenen Endes stellt eine zusätzliche geometrische Annahme dar. Die tatsächliche Oberfläche an der Schnittstelle wird durch eine neu erzeugte Fläche approximiert. Die tatsächliche Schnittfläche muss jedoch nicht exakt planar sein.

17.7 Geschlossenheit des Meshes

Eine Volumenberechnung setzt einen geschlossenen Körper voraus. Bei einem Mesh mit offenen Kanten wird dieses Problem dadurch behandelt, dass die vorhandene Randkurve erkannt und mit einer Deckelfläche verschlossen wird. Bei Schnittmessungen wird die Schnittfläche ebenfalls explizit erzeugt.

18 Zusammenfassung der Verfahren

Die verschiedenen Messmethoden können wie folgt zusammengefasst werden:

Messung	Mathematische Grundlage
Oberflächenstrecke	Geodätischer Pfad und diskrete Approximation der Kurvenlänge
Flächeninhalt	Summe der Flächen der einzelnen Mesh-Dreiecke
Umfang	Summe der euklidischen Abstände benachbarter Punkte eines geschlossenen Polygonzuges
Gesamtvolumen	Volumen eines geschlossenen Polygonnetzes nach vorherigem Verschließen eines offenen Endes
Volumen ab Markierung	Mittelwert der markierten z -Koordinaten und anschließender horizontaler Schnitt
Volumen ab Ring	PCA zur Bestimmung der Ringebene und anschließender geschlossener Schnittkörper
Volumen ab Zwischenfingerfalte	Schnitt mit der festgelegten Referenzebene $z = 0$ und anschließende Erzeugung der Schnittfläche

Tabelle 2: Übersicht über die verwendeten Vermessungsverfahren

19 Fazit

Die implementierten Vermessungsverfahren ermöglichen es, verschiedene geometrische Eigenschaften eines dreidimensionalen Fingerscans direkt aus dem erzeugten Mesh zu bestimmen.

Die Streckenmessung berücksichtigt die Krümmung der Fingeroberfläche, da eine geodätische Strecke anstelle einer einfachen räumlichen Distanz verwendet wird. Die farbbasierte Markierung ermöglicht eine intuitive Definition von Messstellen durch die Verbindung von Texturkoordinaten, RGB-Farbvergleich und dreidimensionalen Vertexkoordinaten.

Bei Ringmarkierungen wird die Orientierung mittels PCA aus den markierten Punkten bestimmt. Die Verwendung der kleinsten Hauptkomponente als Ebenennormale folgt direkt aus der Minimierung des quadratischen Abstands der Punktwolke zu einer Ebene.

Für die Volumenberechnung wird berücksichtigt, dass ein realer 3D-Scan an seinem offenen Ende keine geschlossene Oberfläche besitzen muss. Das aktuelle Verfahren erkennt einen vorhandenen offenen Rand und erzeugt daraus einen geometrisch bestimmten Deckel. Bei Schnittmessungen werden die Schnittflächen ebenfalls explizit erzeugt.

Die Genauigkeit der Ergebnisse ist durch die Qualität des zugrunde liegenden Scans und des daraus erzeugten Meshes begrenzt. Unter der Voraussetzung eines geeigneten und ausreichend fein aufgelösten Meshes stellen die implementierten Verfahren eine nachvollziehbare numerische Approximation der entsprechenden geometrischen Größen dar.